

Universidad Galileo	Guatemala 14 de Septiembre del 2026
Facultad: FISICC	Alumno: Roberto Molina
Curso: Microprocesadores	Carnet: 24000849
Sección: A	Hora de Laboratorio: 1:00 a 1:50
Auxiliar: Brandon Mendoza	Día de Laboratorio: Lunes

Laboratorio #5 Getting Started with Nucleo-G431KB

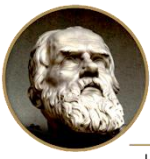
Resumen:

En esta práctica de laboratorio se trabajó con la tarjeta Nucleo-G431KB, utilizando Visual Studio Code y lenguaje C para desarrollar, compilar, cargar y depurar una aplicación de sistemas embebidos. Inicialmente se reconocieron los principales elementos de hardware de la tarjeta, entre ellos el microcontrolador STM32G431KBT6, la interfaz ST-LINK, el LED de usuario y el botón de reset. Posteriormente, se configuró el proyecto en VS Code, se agregaron los archivos main.c, LEDs.c y LEDs.h, y se seleccionó la interfaz ST-Link@pyOCD para programar y depurar la tarjeta.

La aplicación permitió controlar el LED de usuario LD2 mediante la rutina de interrupción SysTick_Handler, observando inicialmente un parpadeo cercano a 5 Hz y modificando posteriormente la frecuencia a 1 Hz, 10 Hz, 20 Hz y 50 Hz para analizar los cambios en su comportamiento. También se utilizó el depurador de VS Code para observar variables mediante Watch, ejecutar el programa paso a paso, trabajar con breakpoints y analizar el comportamiento de variables como ticks, data y copy, incluyendo el efecto de utilizar o eliminar la palabra reservada static.

Objetivos:

- Familiarizarse con la tarjeta Nucleo-G431KB, identificando sus principales componentes y comprendiendo el procedimiento básico para desarrollar una aplicación de sistemas embebidos en lenguaje C.
- Configurar, compilar y cargar un proyecto en Visual Studio Code, utilizando la interfaz ST-Link para programar el microcontrolador STM32G431KBT6 y controlar el LED de usuario de la tarjeta.
- Utilizar las herramientas de depuración de VS Code, como Watches y breakpoints, para observar el comportamiento de variables y analizar la ejecución del programa durante el funcionamiento de la aplicación



Teoría:

Tarjeta NUCLEO-G431KB

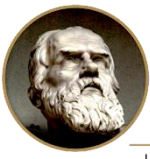
La tarjeta NUCLEO-G431KB es una placa de evaluación de STMicroelectronics diseñada para el desarrollo rápido de aplicaciones embebidas. Según la página oficial del producto, se trata de "an affordable and flexible way for users to try out new concepts and build prototypes" [1] con la familia de microcontroladores STM32. La placa integra un programador/depurador ST-LINK en la misma tarjeta, por lo que "the STM32 Nucleo-32 board does not require any separate probe" [1]; este ST-LINK ofrece, además, "USB re-enumeration capability: mass storage, Virtual COM port, and debug port" [1]. En cuanto a periféricos de usuario, el fabricante indica que la tarjeta cuenta con "1 user LED" y "1 reset push-button" [1], además de un "Micro-AB USB connector for the ST-LINK" [1] que provee tanto la alimentación como la comunicación con la PC, y de un conector de expansión "ARDUINO® Nano V3" [1] compatible con shields.

Microcontrolador STM32G431KBT6

El microcontrolador montado en la tarjeta pertenece a la familia STM32G4 de STMicroelectronics. De acuerdo con la ficha técnica del producto, "the STM32G431x6/x8/xB devices are based on the high-performance Arm® Cortex®-M4 32-bit RISC core" [2], con una frecuencia de operación de hasta 170 MHz y un rendimiento de 213 DMIPS gracias a un acelerador ART que permite ejecución sin estados de espera desde la memoria Flash. El dispositivo dispone de 128 Kbytes de memoria Flash y 22 Kbytes de SRAM, e integra una unidad de punto flotante (FPU), instrucciones DSP y una unidad de protección de memoria (MPU), además de periféricos analógicos (ADCs de 12 bits, DACs, comparadores) y múltiples interfaces de comunicación (I2C, SPI, USART, USB, FDCAN) [2].

Interfaz ST-LINK

El ST-LINK es la herramienta que permite programar y depurar el microcontrolador desde la PC. Según STMicroelectronics, "the ST-LINK is an in-circuit debugger and programmer for the STM8 and STM32 microcontroller families" [3], y se comunica con el microcontrolador mediante las interfaces JTAG/SWD, a través de una conexión USB. Esta herramienta es compatible con múltiples entornos de desarrollo, entre ellos Keil, IAR y las herramientas propias de ST [3]. En el caso de esta práctica, la variante integrada en la tarjeta corresponde a la generación ST-LINK-V3E, que permite tanto cargar el ejecutable en el microcontrolador como depurarlo directamente desde VSCode.



Entorno de desarrollo: Visual Studio Code y Keil Studio (CMSIS-solution)

Para el desarrollo del proyecto se utilizó Visual Studio Code junto con la extensión Arm Keil Studio, la cual habilita el flujo de trabajo basado en CMSIS. Esta herramienta es descrita como "a comprehensive software development platform for Arm Cortex-M processor-based devices" [4], que permite crear proyectos, compilarlos con base en CMSIS y otros frameworks, y depurarlos directamente desde el editor. La documentación indica que la extensión "utilizes CMSIS-Pack content to configure debug adapters and access reusable software components" [4], lo cual explica el uso de archivos como el csolution y la selección del adaptador de depuración (en este caso, ST-Link@pyOCD) para establecer la comunicación entre VSCode y la tarjeta.

Temporizador SysTick

El programa desarrollado en la práctica hace uso de la rutina de interrupción SysTick_Handler para controlar el parpadeo del LED. El SysTick es un temporizador integrado en el núcleo de los procesadores Cortex-M que "generates interrupt requests on a regular basis" [5], y que puede emplearse tanto para la conmutación de tareas en un sistema operativo como para el conteo de tiempo en aplicaciones sin sistema operativo. Su configuración se realiza mediante la función SysTick_Config(ticks), la cual "initialises and starts the System Tick Timer and its interrupt" [5] con base en la cantidad de ciclos de reloj indicada, generando así interrupciones periódicas que, en esta práctica, se utilizan para alternar el estado del LED a distintas frecuencias.

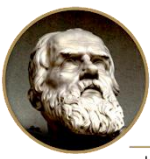
La palabra reservada static en C

Durante la práctica se analizó el efecto de declarar una variable como static dentro de una función. De acuerdo con la referencia del lenguaje C, una variable local declarada con el especificador static adquiere "static storage duration" [6] en lugar de duración automática, lo que significa que "the variable is initialized once when the function is first called" y que "the value persists between function calls" [6]. Esto contrasta con una variable automática (sin static), cuyo valor se pierde al finalizar cada llamada a la función, ya que su memoria se reserva únicamente durante la ejecución de dicha llamada. Este comportamiento es precisamente el que se puso a prueba al observar, mediante el depurador, las diferencias entre las variables ticks, data y copy dentro de la rutina SysTick_Handler.

Resultados prácticos:

```
h LEDs.h X
G4_Test1 > h LEDs.h
1  /* Universidad Galileo
2   * Turing Research Lab
3   * Julio E. Fajardo
4   * Microprocesadores 2020
5   * LED Driver
6   * Aug-08-2020
7   * LEDs.h
8   */
9
10 #ifndef LEDS_H_
11 #define LEDS_H_
12
13 #include <stdint.h>
14
15 void LEDs_Init(void);
16 void LED_On(void);
17 void LED_Off(void);
18 void LED_Toggle(void);
19
20 #endif /* LEDS_H_ */
21
```

Fig. 1. LEDs.h

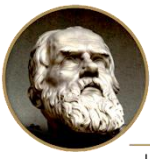


```
G4_Test1 > C LEDs.c
1  /* Universidad Galileo
2   * Turing Research Lab
3   * Julio E. Fajardo
4   * CMSIS-DSP Application
5   * LED Driver
6   * Aug-08-2020
7   * LEDs.c
8   */
9
10 #include "RTE_Components.h"
11 #include CMSIS_device_header
12 #include "LEDs.h"
13
14 /* 0 - STM32F3-EVALBOARD
15     1 - STM32F3-DISCOVERY
16     2 - NUCLEO-F303K8
17     3 - NUCLEO-F303RE
18     4 - NUCLEO-G431KB
19 */
20
21 #define TARGET 4
22
23 const uint32_t leds_mask[] = { 1<<13, 1<<13, 1<<3, 1<<5, 1<<8 };
24
```

Fig. 2. Declaración en LEDs.c

```
// LEDs Initialization
void LEDs_Init(void){
    #if ((TARGET == 0) || (TARGET == 1))
        RCC->AHBENR |= RCC_AHBENR_GPIOEEN;    // GPIOE Clock Enable
        GPIOE->MODER |= (1<<26);
        GPIOE->ODR &= ~(1<<13);
    #elif (TARGET == 2)
        RCC->AHBENR |= RCC_AHBENR_GPIOBEN;    // GPIOB Clock Enable
        GPIOB->MODER &= ~(1<<7);
        GPIOB->MODER |= (1<<6);
        GPIOB->ODR &= ~(1<<9);
    #elif (TARGET == 3)
        RCC->AHBENR |= RCC_AHBENR_GPIOAEN;    // GPIOE Clock Enable
        GPIOA->MODER |= (1<<10);
        GPIOA->ODR &= ~(1<<5);
    #elif (TARGET == 4)
        RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;    // GPIOB Clock Enable
        GPIOB->MODER &= ~(1<<17);
        GPIOB->MODER |= (1<<16);
        GPIOB->ODR &= ~(1<<8);
    #endif
}
```

Fig. 3. Función LEDs_Init en LEDs.c



```
// Turn OFF LED, receives as argument the led number (0 is G, 1 is Y, 2 is R and 3 is B)
void LED_Off(void){
    #if((TARGET==0)|| (TARGET==1))
        GPIOE->ODR |= leds_mask[TARGET];
    #elif (TARGET==2)
        GPIOB->ODR |= leds_mask[TARGET];
    #elif (TARGET==3)
        GPIOA->ODR |= leds_mask[TARGET];
    #elif (TARGET==4)
        GPIOB->ODR &= ~leds_mask[TARGET];
    #endif
}

// Turn ON LED, receives as argument the led number (0 is G, 1 is Y, 2 is R and 3 is B)
void LED_On(void){
    #if((TARGET==0)|| (TARGET==1))
        GPIOE->ODR &= ~leds_mask[TARGET];
    #elif (TARGET==2)
        GPIOB->ODR &= ~leds_mask[TARGET];
    #elif(TARGET==3)
        GPIOA->ODR &= ~leds_mask[TARGET];
    #elif(TARGET==4)
        GPIOB->ODR |= leds_mask[TARGET];
    #endif
}
```

Fig. 4. Función LED_Off y LED_On en LEDs.c

```
// LED Toggle, receives as argument the led number (0 is G, 1 is Y, 2 is R and 3 is B)
void LED_Toggle(void){
    #if((TARGET==0)|| (TARGET==1))
        GPIOE->ODR ^= leds_mask[TARGET];
    #elif(TARGET==2)
        GPIOB->ODR ^= leds_mask[TARGET];
    #elif(TARGET==3)
        GPIOA->ODR ^= leds_mask[TARGET];
    #elif(TARGET==4)
        GPIOB->ODR ^= leds_mask[TARGET];
    #endif
}
```

Fig. 5. Función LED_Toggle en LEDs.c

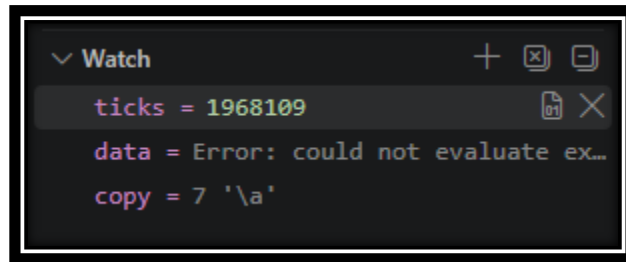


Fig. 6. Watch

Video demostrativo: https://youtube.com/shorts/8iQwG_PNwOg?feature=share

Procedimientos teóricos:

Archivo LEDs.h

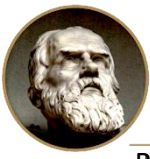
El archivo LEDs.h (Fig. 1) contiene las cabeceras de las funciones del driver del LED: LEDs_Init(void), LED_On(void), LED_Off(void) y LED_Toggle(void). Al ser un archivo de encabezado, no implementa la lógica de estas funciones, sino que únicamente las declara para que puedan ser utilizadas desde main.c mediante la directiva #include "LEDs.h", protegida además por las directivas #ifndef / #define / #endif para evitar inclusiones múltiples del mismo archivo.

Configuración del proyecto y leds_mask en LEDs.c

Al inicio de LEDs.c (Fig. 2) se definen las cabeceras necesarias (RTE_Components.h, el header del dispositivo CMSIS y LEDs.h), y se declara la macro TARGET, la cual permite reutilizar el mismo driver para distintas tarjetas de desarrollo (STM32F3-EVALBOARD, STM32F3-DISCOVERY, NUCLEO-F303K8, NUCLEO-F303RE y NUCLEO-G431KB) simplemente cambiando su valor; en esta práctica TARGET se definió como 4, correspondiente a la NUCLEO-G431KB. Justo debajo se declara el arreglo leds_mask[], que almacena, para cada tarjeta soportada, la máscara de bits asociada al pin del puerto GPIO donde está conectado el LED de usuario, de forma que el resto de funciones del driver puedan operar sobre el pin correcto simplemente indexando este arreglo con TARGET.

Función LEDs_Init

La función LEDs_Init() (Fig. 3) configura el pin del LED como salida digital mediante una serie de directivas #if / #elif que seleccionan, en tiempo de compilación, el bloque de código correspondiente al valor de TARGET. Para el caso de la NUCLEO-G431KB (TARGET == 4), la función primero habilita el reloj del puerto GPIOB escribiendo sobre el registro



RCC->AHB2ENR (bit GPIOBEN), y después configura el pin del LED como salida limpiando y luego estableciendo los bits correspondientes del registro GPIOB->MODER, finalizando

con la limpieza del bit correspondiente en GPIOB->ODR para dejar el LED apagado como estado inicial.

Funciones LED_Off y LED_On

Estas dos funciones (Fig. 4) manipulan directamente el registro ODR (Output Data Register) del puerto GPIO correspondiente usando la máscara leds_mask[TARGET]: LED_Off() realiza un AND con el complemento de la máscara (&= ~leds_mask[TARGET]) para poner en cero el bit del LED, apagándolo, mientras que LED_On() realiza un OR con la máscara (|= leds_mask[TARGET]) para poner en uno ese mismo bit, encendiéndolo. Al igual que en LEDs_Init, la operación exacta depende del bloque #if/#elif seleccionado según TARGET.

Función LED_Toggle

LED_Toggle() (Fig. 5) invierte el estado actual del LED mediante una operación XOR sobre el registro ODR (GPIOB->ODR ^= leds_mask[TARGET] para TARGET == 4), de modo que si el bit estaba en 0 pasa a 1 y viceversa. Esta es la función que efectivamente produce el parpadeo del LED, ya que se invoca en cada interrupción del temporizador SysTick.

¿Qué hacen las funciones main y SysTick_Handler?

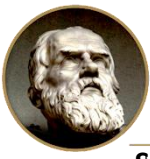
En main() se llama primero a LEDs_Init() para configurar el pin del LED como salida, y luego a SysTick_Config(SystemCoreClock/FREQ), que programa el temporizador SysTick para generar una interrupción cada SystemCoreClock/FREQ ciclos de reloj. El programa permanece después en un ciclo while(1). Cada vez que ocurre la interrupción se ejecuta automáticamente SysTick_Handler(), la cual incrementa la variable local data y llama a LED_Toggle(), alternando el estado del LED. En conjunto, ambas funciones son las responsables del parpadeo del LED a la frecuencia definida por FREQ.

El LED parpadea a 5 Hz, ¿ya sabe qué hace la aplicación?

Sí: la aplicación alterna el estado del LED de usuario en cada interrupción generada por el SysTick, configurado inicialmente con FREQ = 5 mediante SysTick_Config(SystemCoreClock/FREQ).

Se modificó la frecuencia a 1 Hz, ¿se nota la diferencia?

Sí, el parpadeo se vuelve claramente más lento en comparación con el observado a 5 Hz.



Se modificó la frecuencia a 10 Hz, 20 Hz y 50 Hz

El parpadeo se vuelve cada vez más rápido conforme aumenta FREQ, hasta el punto en

que a 50 Hz resulta difícil de percibir a simple vista, debido a la persistencia visual del ojo humano.

Se agregó data al Watch después de ticks, ¿qué ocurrió? ¿por qué?

El Watch mostró un error indicando que la expresión no podía evaluarse (Fig. 6). Esto ocurrió porque, aunque data está declarada como static y por lo tanto conserva su valor entre interrupciones sucesivas, su alcance (scope) sigue siendo local a la función SysTick_Handler; mientras el programa está detenido en un punto donde esa función no está activa en la pila de llamadas (por ejemplo, dentro del while(1) de main), el depurador no puede resolver ese identificador.

Se declaró la variable global copy y se agregó al Watch, ¿qué ocurrió? ¿por qué?

A diferencia de data, copy sí se pudo visualizar correctamente en cualquier punto de la ejecución (Fig. 6 muestra copy = 7 '\a'), ya que al ser una variable global su alcance cubre todo el programa y es accesible desde el contexto en que el depurador evalúa el Watch.

¿Qué ocurre si se le quita la palabra reservada static a la variable data?

Al quitar static, data deja de tener duración de almacenamiento estática y pasa a comportarse como una variable automática (se crea y se destruye en la pila cada vez que se invoca SysTick_Handler). Como resultado, ya no conserva su valor entre una interrupción y la siguiente, por lo que data++ deja de acumular un conteo real.

Se agregó un breakpoint en la línea que escribe a los LEDs y se ejecutó de forma continua, ¿qué ocurrió?

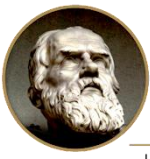
La ejecución se detuvo automáticamente cada vez que el flujo llegó a la línea LED_Toggle();, es decir, en cada disparo de la interrupción del SysTick, permitiendo inspeccionar el estado de las variables justo antes de actualizar el LED.

Se ejecutó el código nuevamente, ¿qué ocurrió?

El programa avanzó hasta el siguiente ciclo de interrupción del SysTick y volvió a detenerse en el mismo breakpoint, repitiéndose este comportamiento en cada nueva interrupción.

¿La aplicación hace un buen uso de las variables? ¿Habría que modificar algo?

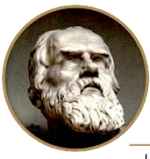
No completamente: si bien data sí requiere el calificador static para persistir su valor entre interrupciones, cualquier variable global modificada dentro de una rutina de interrupción



(ISR) y leída desde el programa principal como copy, debería declararse además como volatile, ya que de lo contrario el compilador podría optimizar su acceso asumiendo que el valor no cambia fuera del flujo normal del programa, generando lecturas incorrectas o desactualizadas cuando la ISR la modifica de forma asíncrona.

Conclusiones:

- Esta práctica resultó bastante interesante, ya que fue la primera vez que se trabajó directamente con un microcontrolador físico. Al inicio generó cierto nerviosismo conectar y manipular la tarjeta NUCLEO-G431KB, principalmente por temor a dañarla, pero conforme avanzó la práctica se ganó confianza en el manejo del hardware y en el proceso de programación y depuración.
- Se comprendió de forma práctica la diferencia entre el alcance (scope) y la duración de almacenamiento de una variable, algo que en teoría parecía sencillo, pero que al observarlo directamente en el depurador al no poder visualizar data desde fuera de SysTick_Handler, y sí poder hacerlo con la variable global copy permitió entender de una manera mucho más clara por qué estos conceptos son importantes al trabajar con interrupciones en sistemas embebidos.
- Trabajar a nivel de registros (GPIO, RCC, SysTick) en lugar de usar funciones de más alto nivel permitió apreciar mejor lo que realmente ocurre por debajo cuando se enciende un LED o se configura un temporizador, lo cual resultó útil para entender el funcionamiento interno de un microcontrolador y no solo depender de librerías que ocultan esos detalles.



Bibliografía:

- [1] "NUCLEO-G431KB | Product - STMicroelectronics". STMicroelectronics. Accedido el 18 de septiembre de 2026. [En línea]. Disponible: <https://www.st.com/en/evaluation-tools/nucleo-g431kb.html>
- [2] "STM32G431KB | Product - STMicroelectronics". STMicroelectronics. Accedido el 18 de septiembre de 2026. [En línea]. Disponible: https://www.st.com/content/st_com/en/products/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus/stm32-mainstream-mcus/stm32g4-series/stm32g4x1/stm32g431kb.html
- [3] "ST-LINK | Tool - STMicroelectronics". STMicroelectronics. Accedido el 18 de septiembre de 2026. [En línea]. Disponible: <https://www.st.com/en/development-tools/st-link.html>
- [4] "Arm Keil Studio for VS Code". Site not found · GitHub Pages. Accedido el 18 de septiembre de 2026. [En línea]. Disponible: <https://mdk-packs.github.io/vscode-cmsis-solution-docs/>
- [5] "CMSIS-Core (Cortex-M): SysTick Timer (SYSTICK)". Site not found · GitHub Pages. Accedido el 18 de septiembre de 2026. [En línea]. Disponible: https://arm-software.github.io/CMSIS_6/main/Core/group_SysTick_gr.html
- [6] "Static storage duration - cppreference.com". cppreference.com. Accedido el 18 de septiembre de 2026. [En línea]. Disponible: https://en.cppreference.com/c/language/static_storage_duration